Chương 6 · Static Testing

1. Kỹ thuật static testing

Định nghĩa
- Kiểm thử ở mức đặc tả/cài đặt, KHÔNG thực thi phần mềm

Lỗi dễ tìm thấy
- Sai chuẩn (standards)
- Thiếu yêu cầu
- Lỗi thiết kế
- Code khó bảo trì
- Interface đặc tả không nhất quán

Mục tiêu
- Phát hiện lỗi sớm ở mọi pha SDLC
- Xác minh đúng yêu cầu
- Cung cấp thông tin giám sát dự án
- Tăng chất lượng & năng suất

Phân loại

Review
- Informal review
- Walkthrough
- Technical review
- Inspection

Static analysis
- Coding standards
- Code metrics
- Code structure

2. Review process

Khái niệm
- Quá trình/cuộc họp kiểm tra tài liệu bởi ≥1 người
- Mục tiêu: tìm lỗi + thông tin, trao đổi, đào tạo

6 pha formal review

① Planning
- Ai làm: MODERATOR (chọn phần review, lập team, lên lịch, entry check)
- Gửi 'request for review' cho moderator (slide không nói rõ ai gửi, thường là Author)

Entry check
- Mẫu ~30 phút: ≤3 major defect/trang
- Tài liệu có số dòng, đã dọn sạch
- Tham chiếu ổn định, sẵn có

- Chọn phần cần review
- Team 4–6 người, nhiều vai trò
- Lên lịch họp, địa điểm

② Kick-off
- Ai làm: MODERATOR tổ chức, AUTHOR giải thích tài liệu (suy từ bảng vai trò)
- Tùy chọn nhưng rất nên có
- Thống nhất cách hiểu: phát tài liệu, giải thích mục tiêu
- Họp hoặc gửi mail đều được
- Hữu ích cho dự án mới/phức tạp

③ Preparation
- Ai làm: REVIEWER / mọi người tham gia, mỗi người tự làm
- Mỗi người tự làm trước họp

- Dùng checklist, quy trình, tài liệu liên quan
- Ghi lại mọi issue
- Checking rate 5–10 trang/giờ

④ Review meeting

- Ai làm: MODERATOR dẫn dắt cả buổi họp
- Tối đa 2 giờ

Logging

- Ai làm: SCRIBE ghi, REVIEWER nêu lỗi
- Ghi từng trang, từng reviewer
- Gán severity: critical / major / minor
- KHÔNG thảo luận

Discussion

- Ai làm: AUTHOR, REVIEWER thảo luận; MODERATOR kiểm soát
- Trình bày ý kiến & lý do
- Moderator xử lý vấn đề con người

Decision

- Ai làm: các thành viên cùng quyết định; MODERATOR đóng họp
- Quyết định theo exit criteria
- VD: số lỗi critical/major TB mỗi trang
- Moderator đóng họp

⑤ Rework

- Ai làm: AUTHOR
- Sửa tài liệu theo action items
- Thay đổi phải dễ nhận ra
- Không phải lỗi nào cũng sửa
- Không sửa thì phải báo cáo

⑥ Follow-up

- Ai làm: MODERATOR chủ trì cùng AUTHOR
- Họp lại sau rework
- Kiểm tra lỗi đã xử lý, exit criteria
- Quyết định đạt hay review lại
- Gửi bản mới cho mọi người, lấy feedback
- Thu thập metrics

Vai trò

Author

- Giải thích tài liệu, trả lời câu hỏi
- Sửa lỗi, cung cấp bản đã sửa

Moderator

- Chuẩn bị: agenda, tài liệu, checklist
- Thu feedback, điều hành họp
- Ra báo cáo, theo dõi đến khi sửa xong

Reviewer

- Đọc tìm lỗi, gửi feedback, dự họp

Secretary/Scribe (tùy chọn)

- Ghi biên bản, hỗ trợ moderator

Project members (tùy chọn)

- Hỗ trợ trả lời, dự họp

Metrics (inspection)

- Review rate: số trang review/giờ
- Review effort: thời gian review 1 trang
- Defect finding rate: số lỗi tìm được/giờ
- Defect finding effort: số lỗi/trang (theo slide)

Success factors
- Xem ebook trang 70

3. Static analysis (dùng tool)

Khái niệm
- Phân tích chương trình KHÔNG thực thi (BS 7925-1)
- Tìm: unreachable code, data flow anomaly, infinite loop
- Thường dùng tool hỗ trợ

Coding standards

Mục đích
- Riêng cho từng ngôn ngữ
- ~80% chi phí vòng đời là bảo trì
- Tăng khả năng đọc, hiểu code nhanh

Vì sao dùng tool
- Quá nhiều rule để nhớ
- Rule nhạy ngữ cảnh qua nhiều file khó kiểm tay
- Check tay làm review kém hiệu quả

Quy ước phổ biến
- Indent 4 space, tránh Tab
- Dòng ≤ 80/120 ký tự
- Blank line / blank space đúng chỗ
- Comment, số khai báo mỗi dòng

Naming
- Có nghĩa, ≤20 ký tự, tránh viết tắt
- Class: danh từ
- Biến/method: bắt đầu chữ thường (method dùng động từ)
- Hằng: CHỮ HOA có _

Code metrics
- Comment frequency
- Depth of nesting
- Number of lines of code
- Cyclomatic number

Code structure

Control flow
- Tìm infinite loop, multiple entry, dead code
- Biểu diễn bằng CFG
- Đo nested level, cyclomatic complexity

Data flow
- Đi từ define (gán) → use (đọc)
- Lỗi: biến không dùng, dùng khi chưa định nghĩa, gán sai
- Anomaly: định nghĩa lại khi chưa dùng

Data structure
- Phân tích cấu trúc dữ liệu: list, queue, stack
- Cho biết độ khó viết code & thiết kế test case

Giá trị mang lại
- Phát hiện lỗi sớm trước khi chạy test
- Cảnh báo sớm điểm đáng ngờ
- Tìm lỗi khó thấy khi dynamic test
- Code/thiết kế dễ bảo trì hơn
- Phòng ngừa lỗi

4. Self-code review

Là gì

- Dev tự review khi code
- Logic đúng yêu cầu
- Không vi phạm convention/lỗi phổ biến
- Áp dụng practice chung

Cách làm

- Công cụ: FxCop, Resharper, StyleCop (.NET), CheckStyle (Java), CPPCheck (C/C++)
- Checklist do team định nghĩa, điền kết quả

Lỗi thường gặp

Hard-code constant

- Khó đổi nhiều chỗ → đưa vào module hằng/file config

Index mảng C từ 0

- a[10] chỉ có 0..9; vòng i<=10 sai

Dangling else

- else gắn với if gần nhất → dùng { }

Null pointer

- Kiểm tra null trước khi dùng

SQL injection, memory leak

- Nối chuỗi SQL, quên dispose/free

Khai báo trong vòng lặp

- Cấp phát lặp → khai báo trước vòng lặp

Code thừa/dư

- Biến, method không dùng; tạo object không cần

Khác

- Dùng class name gọi static
- Không hard-code số (trừ -1,0,1)
- Không gán lồng/gán nhiều biến một câu
- Ngoặc () cho biểu thức hỗn hợp
- Giải phóng bộ nhớ (C/C++), clear() Collection

Chương 8 · Test Management

0. Vì sao cần test management

- Nhiều test case: cái nào chưa test?
- Kết quả: pass/fail & nguyên nhân?
- Defect: ai chịu trách nhiệm?

1. Tổ chức (Organisation)

Mức độc lập (thấp→cao)

- Chỉ developer
- Nhóm phát triển
- Tester trong dev team
- Đội tester chuyên biệt
- Test consultant nội bộ
- Tổ chức bên ngoài (3rd party)

Ưu / nhược từng mô hình

Developer

- \+ biết code, sửa rẻ
- − khó tự phá, thiên kiến, chủ quan

Dev team

- \+ độc lập phần nào, sâu kỹ thuật
- − áp lực dev, nhìn kỹ thuật, thiếu kỹ năng test

Tester trong dev team

- \+ góc nhìn độc lập, cùng mục tiêu
- − thiếu tôn trọng, cô đơn, dễ bị ảnh hưởng, 1 góc nhìn

Đội test độc lập

- \+ chuyên môn, khách quan, nhất quán
- − bị cô lập, đối đầu, dev ỷ lại tester

Consultant nội bộ

- \+ chuyên sâu, lập kế hoạch/ước lượng tốt
- − đủ chuyên môn? cần kỹ năng giao tiếp

Bên ngoài

- \+ chuyên nghiệp, độc lập chính trị nội bộ
- − thiếu hiểu biết công ty/sản phẩm, đắt

Lựa chọn thường gặp

- Component, integration nhỏ: programmer
- System: đội test độc lập
- Acceptance: user (có hỗ trợ kỹ thuật)

Test leader

- Xây dựng objective, policy, strategy, plan
- Ước lượng & thương lượng với quản lý
- Nhận ra khi nào cần automation
- Dẫn dắt phân tích/thiết kế/thực thi test
- Quản lý cấu hình testware
- Chuẩn bị môi trường test
- Lên lịch, theo dõi, kiểm soát, báo cáo
- Viết test summary report

Tester

- Review/đóng góp test plan
- Phân tích yêu cầu, thiết kế
- Tạo test condition, test case, test data

- Dựng môi trường
- Thực thi test, so sánh kết quả
- Giám sát, thu thập metric
- Review chéo công việc nhau

Kỹ năng

- Chuyên môn nghiệp vụ
- Công nghệ
- Kiểm thử
- + kỹ năng nghề nghiệp & xã hội
- Cần tester chuyên nghiệp, test nghiệp dư không đủ

2. Test plan & ước lượng

Test plan

- Là kế hoạch dự án cho công việc test
- Không phải test design spec / test case / procedure
- Viết để: định hướng tư duy, giao tiếp với stakeholder, quản lý thay đổi

IEEE 829 (16 mục)

- 1 Identifier, 2 Introduction, 3 Test items
- 4 Features to be tested, 5 Features not to be tested
- 6 Approach, 7 Pass/fail criteria
- 8 Suspension & resumption
- 9 Deliverables, 10 Testing tasks, 11 Environment
- 12 Responsibilities, 13 Staffing & training
- 14 Schedule, 15 Risks & contingencies, 16 Approvals
- Slide chia theo: What? How? Who? When?

Ước lượng (effort: thời gian, chi phí)

Metrics-based

- Dựa dữ liệu dự án trước/tương tự
- Số test condition/case viết, chạy; thời gian viết/chạy; số defect; số lần outage
- Đơn giản → phức tạp: phân loại size × complexity, effort TB/test case, mô hình toán

Expert-based

- Hỏi người làm & chuyên gia
- Business expert, test consultant, dev, architect, analyst, designer

3. Giám sát & kiểm soát

Progress monitoring

- Mục đích: phản hồi cho team/manager, thấy kết quả, đo theo exit criteria, thu dữ liệu ước lượng sau
- Cách: tài liệu/spreadsheet/DB hoặc tool
- Metric: theo test case, % chuẩn bị môi trường, độ phủ (yêu cầu/rủi ro/code/cấu hình), chi phí

Reporting

- Lợi ích: truyền đạt kết quả, hỗ trợ kết luận & quyết định
- Cách: IEEE 829 test summary report, tóm tắt metric/biểu đồ

Test control

- Hành động chỉnh hướng để đạt kết quả tốt nhất
- Ví dụ: ưu tiên lại test khi có rủi ro, đổi lịch, siết entry/exit criteria

4. Rủi ro & testing

Risk

- Khả năng xảy ra kết quả không mong muốn
- = likelihood (0–100%) × impact

Product risk

- Liên quan sản phẩm đang test

- Thiếu chức năng chính, kém tin cậy, gây thiệt hại tài chính, chất lượng kém

Project risk

- Liên quan cách thực hiện
- Nhà cung cấp, tổ chức, kỹ thuật
- VD: giao test item trễ, môi trường test không sẵn sàng, sửa defect quá chậm

Cách xử lý rủi ro

- Mitigation: làm trước
- Contingency: có kế hoạch dự phòng
- Transfer: chuyển cho bên khác
- Ignore: bỏ qua

5. Incident management

Incident

- Mọi sự kiện khi test cần điều tra/sửa
- Kết quả thực ≠ mong đợi (defect), lỗi môi trường, dữ liệu test hỏng, expected result sai, tester nhầm
- Có thể cho cả tài liệu lẫn code

Mục tiêu incident report

- Cho dev/quản lý thông tin chi tiết
- Cho test leader theo dõi chất lượng & tiến độ
- Gợi ý cải tiến quy trình

Nội dung (IEEE 829)

- 1 Identifier, 2 Summary
- 3 Description: input, expected, actual, anomalies, date/time, procedure step, environment, lần lặp lại, ý kiến
- 4 Impact

Severity vs Priority

Severity (tác động tới hệ thống)

- Mission critical: hệ thống hỏng
- Major: nặng nhưng có workaround
- Minor: không ảnh hưởng chức năng/usability, lệch spec

Priority (thứ tự xử lý)

- Immediate: sửa ngay
- Delayed: sửa trước level test/ship tiếp theo
- Deferred: có thể để lại vì thời gian/chi phí

Vòng đời incident (slide 49)

- Reported → Opened → Reviewed
- Approved for repair → Assigned → Fixed → Repaired → Confirmation test → Closed
- Rejected: Not a problem / Bad report → Rewritten
- Deferred; confirm fail → Reopened